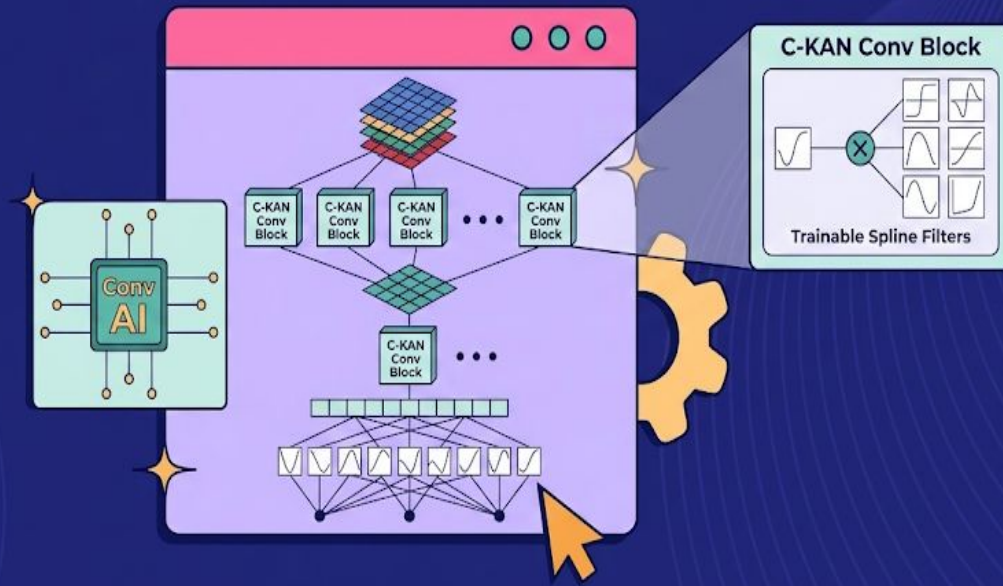
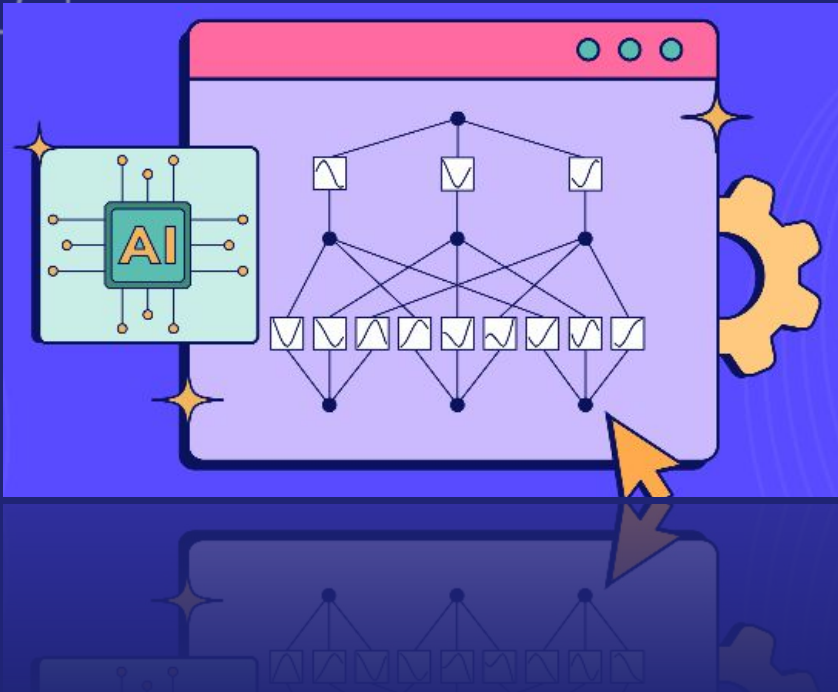




Kolmogorov Arnold Network With Convolutional Network

TEAM

- **Anurup Bhowmick (12624010004)**
- **Abhiraj Sardar (12624010001)**
- **Devang Kamdar (12624010014)**



Contents

1

The What
(Architecture
of KAN)

2

The Why
(Advantages
over MLPs)

3

The How
(Kolmogorov
Arnold
Theorem)

4

Our
CON-KAN
Architecture

5

Experiments
and Results

The What (Architecture)

The **Kolmogorov-Arnold Representation Theorem** states that any multivariate continuous function (a function with multiple inputs) can be expressed as a finite combination of continuous functions of a single variable and addition. In simpler terms, complex multi-input functions can be broken down into simpler one-input functions and summed up, meaning you don't need complex multi-variable interactions for universal function approximation, just enough layers of these simple functions. This theorem is fundamental to the recent development of Kolmogorov-Arnold Networks (KANs), which use these univariate functions as learnable activation functions, offering an alternative to traditional neural networks.

Multivariate
continuous
function

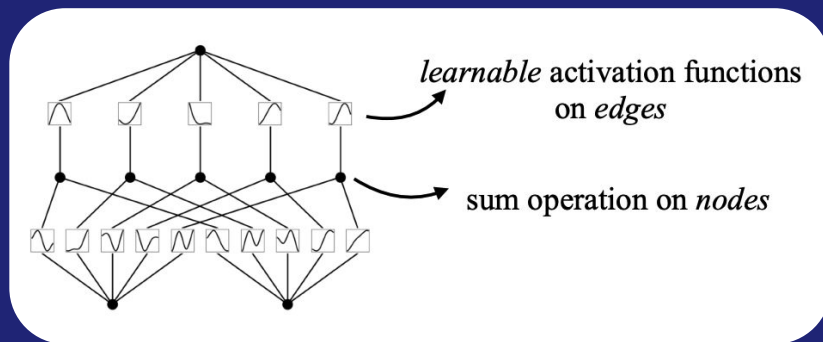
Composition of
univariate functions

$$F(x_1, x_2, x_3, \dots, x_n) = \sum_{j=1}^m \psi_j \left(\sum_{i=1}^n \phi_{ij}(x_i) \right)$$

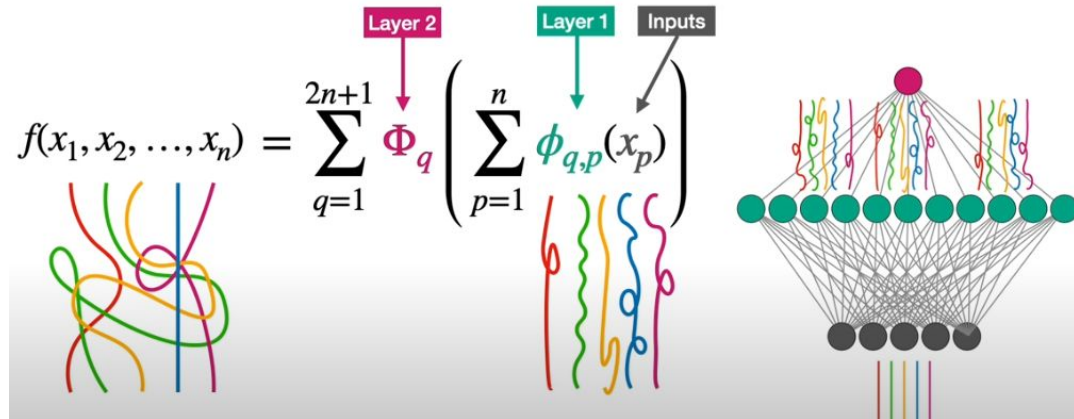
The What (Architecture)

Kolmogorov Arnold Networks (KANs) draw on the Kolmogorov-Arnold representation theorem and offer an alternative to traditional MLPs. Instead of using fixed activations at each neuron, they learn a univariate function on every connection, implemented with splines. This removes the need for linear weights.

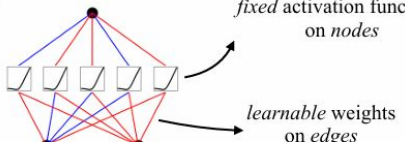
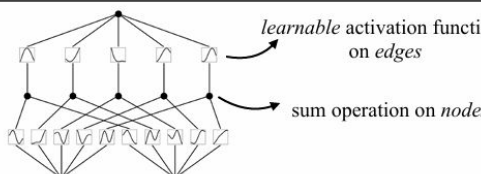
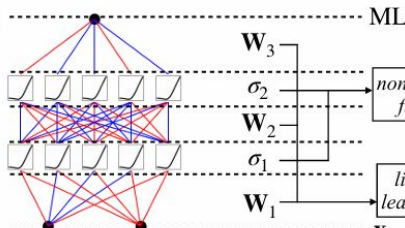
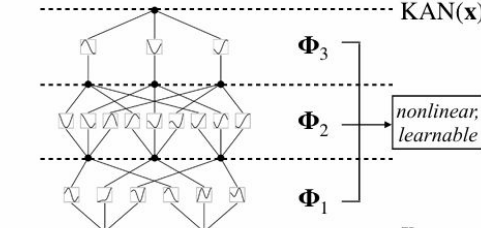
This change helps KANs match or beat MLPs on small scientific tasks with fewer parameters. They show faster scaling behavior and are easier to interpret, since their functions can be visualized and inspected. Examples from math and physics suggest that KANs can support scientific discovery. Overall, KANs provide a promising direction beyond standard MLP architectures.



The What (Architecture)

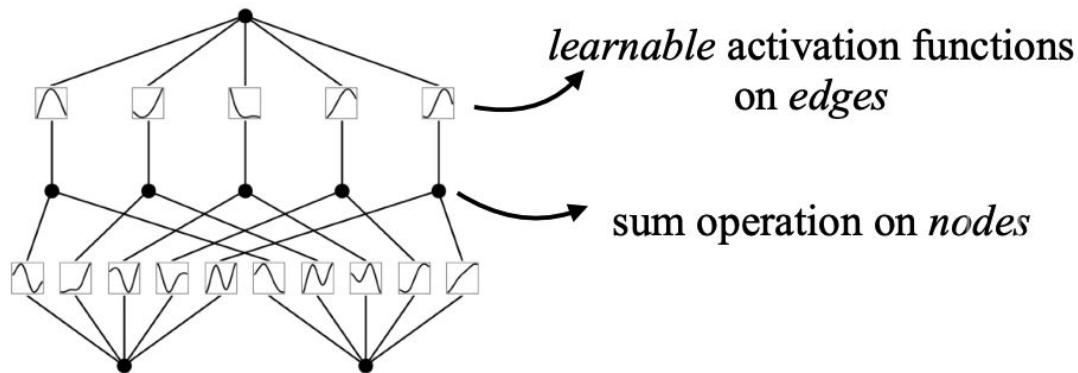


KAN vs MLP

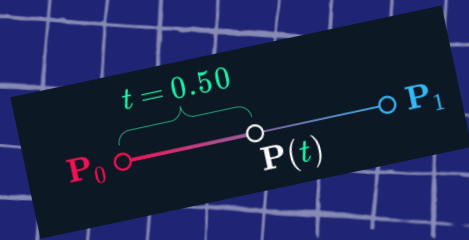
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(e)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a)  fixed activation functions on nodes learnable weights on edges	(b)  learnable activation functions on edges sum operation on nodes
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c)  MLP(x) $\mathbf{W}_3$ σ_2 $\mathbf{W}_2$ σ_1 $\mathbf{W}_1$ $\mathbf{x}$ nonlinear, fixed linear, learnable	(d)  KAN(x) Φ_3 Φ_2 Φ_1 $\mathbf{x}$ nonlinear, learnable

The WHY (Applications)

Interpretability: This is KAN's main advantage. In an MLP, the relationship between input and output is obscured by complex, fixed activation functions within nodes (black box nature). In a KAN, learnable univariate functions are placed on the edges, and these individual functions can be visualized and even set to symbolic forms (e.g., $\sin(x)$, $\exp(y)$), allowing human users to understand how each input variable affects the output.



The How (B-Splines)



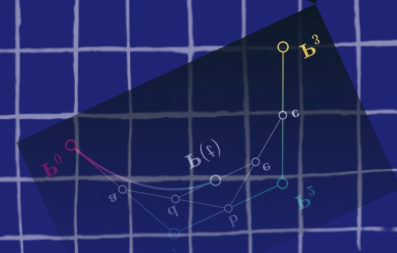
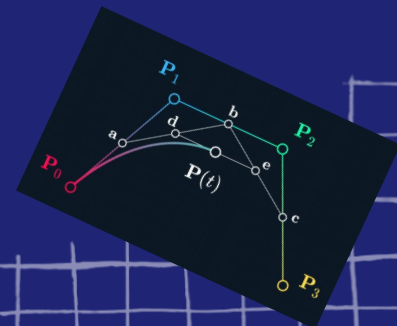
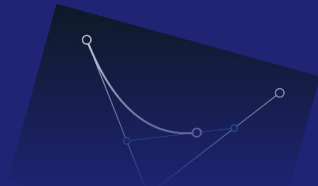
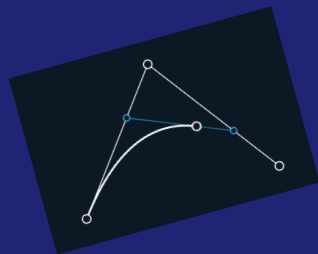
$$P(t) = (1-t)P_0 + tP_1$$

$$P(t) = (1-t)B^0 + tB^1$$



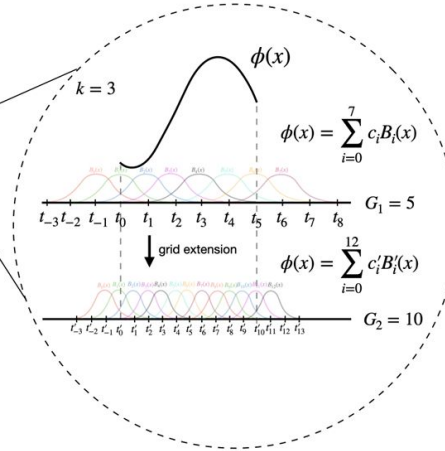
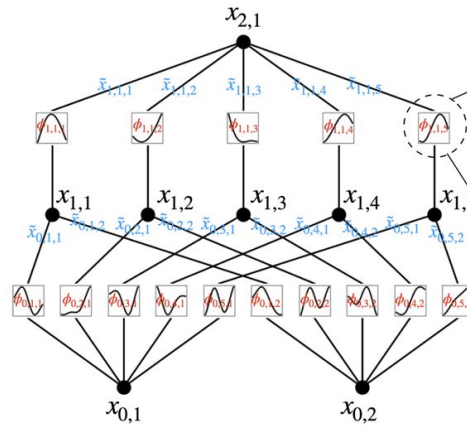
Splines

Bezier



B-Spline Visualizer

The How (B-Splines)



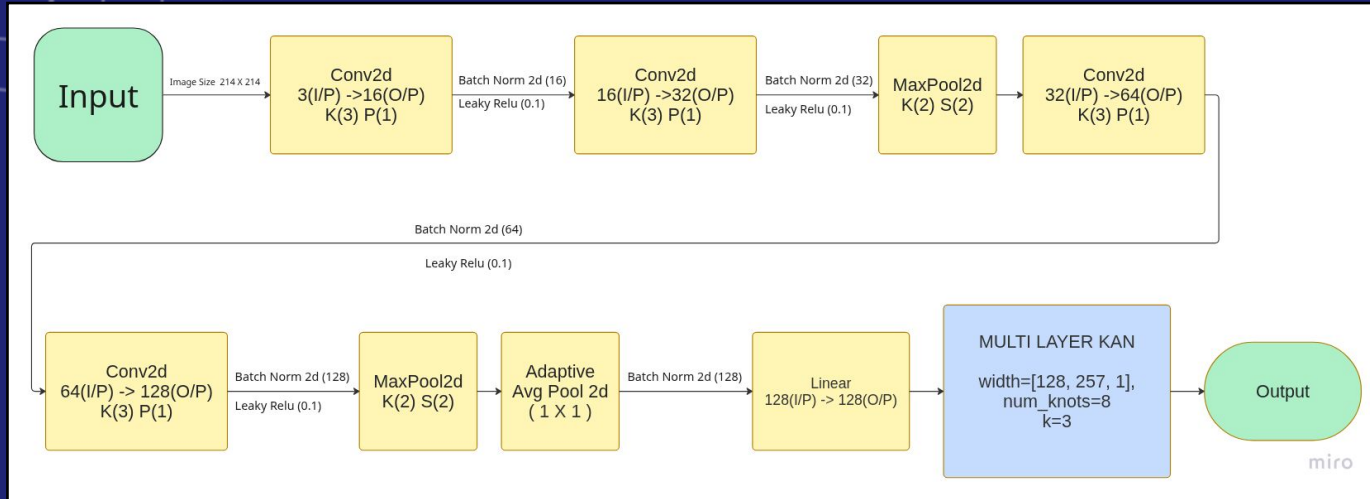
B-Spline Visualizer

Covolutional KAN (CON-KAN)

The CON-KAN architecture is a hybrid deep learning model:

- **It combines CNNs for efficient spatial feature extraction with KANs for adaptive functional classification.**
- **The design replaces the traditional Multi-Layer Perceptron (MLP) classifier with a custom KAN-based classification module.**
- **The overall pipeline consists of convolutional feature extraction, feature normalization, dimensional transformation, and KAN-based classification.**
- **A hybrid approach was necessary due to the high computational cost of full KAN operations in a CPU deployment environment.**
- **The architecture successfully balances experimental research, computational feasibility, and competitive classification performance.**

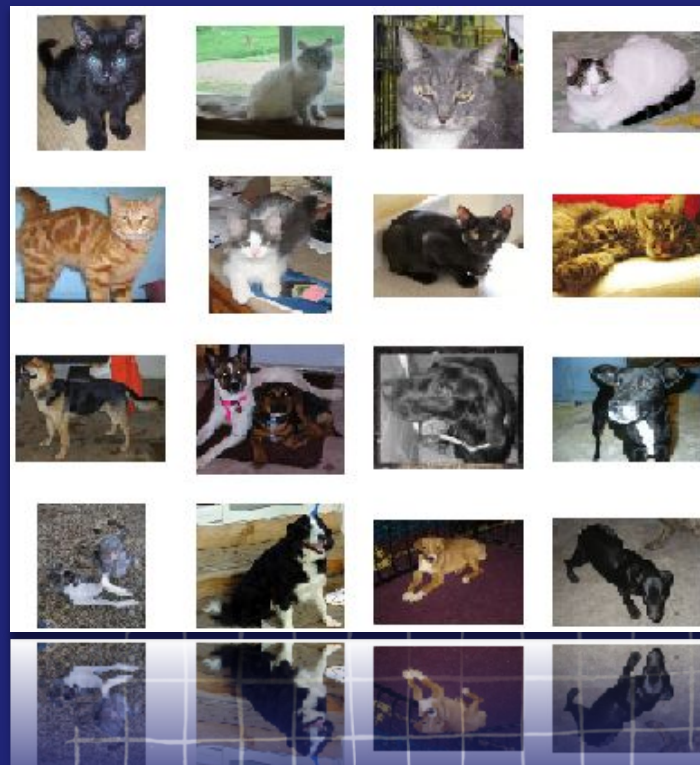
CON-KAN Architecture



- Conv2D (3 -> 16)
- BatchNorm2D
- LeakyReLU
- Conv2D (16 -> 32)
- BatchNorm2D
- LeakyReLU
- MaxPool2D
- Conv2D (32 -> 64)
- BatchNorm2D
- LeakyReLU
- Conv2D (64 -> 128)
- BatchNorm2D
- LeakyReLU
- MaxPool2D
- AdaptiveAvgPool2D

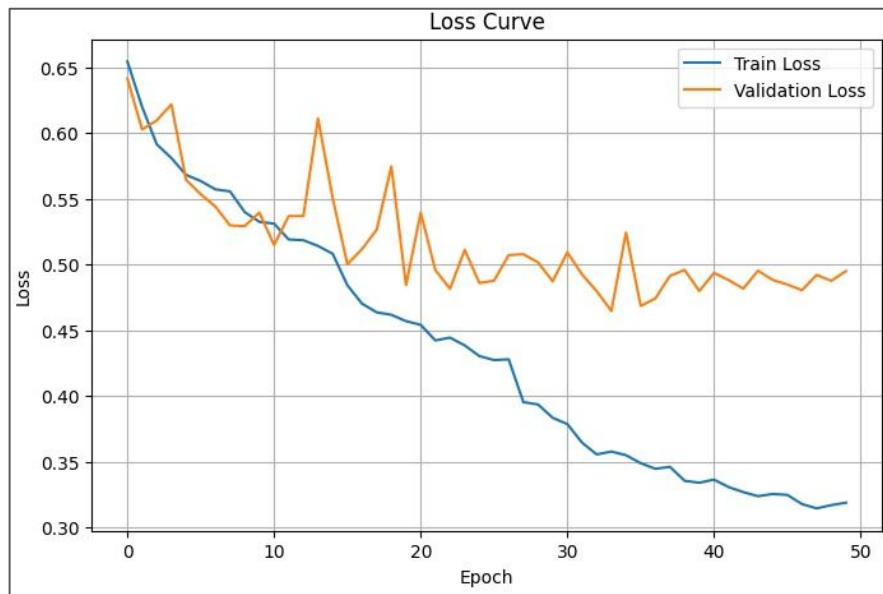
Experiments Performed (RGB image)

We conducted experiments on the Kaggle Cat and Dog dataset containing over 10,000 labeled images for training and testing. The dataset included diverse cat and dog images with different backgrounds, poses, and lighting conditions, helping evaluate the classification performance and learning capability of the proposed CON-KAN architecture.



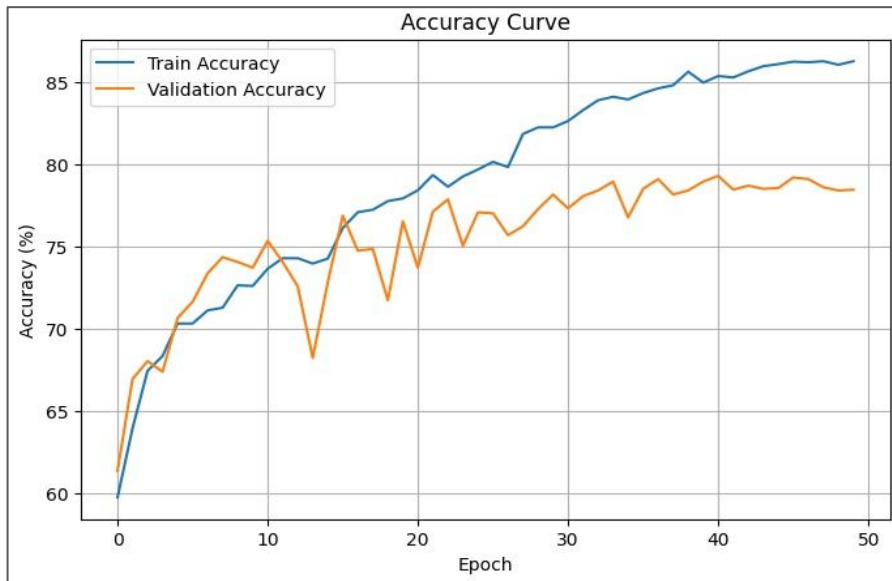
Loss Curve

(RGB image - Cat/Dog)



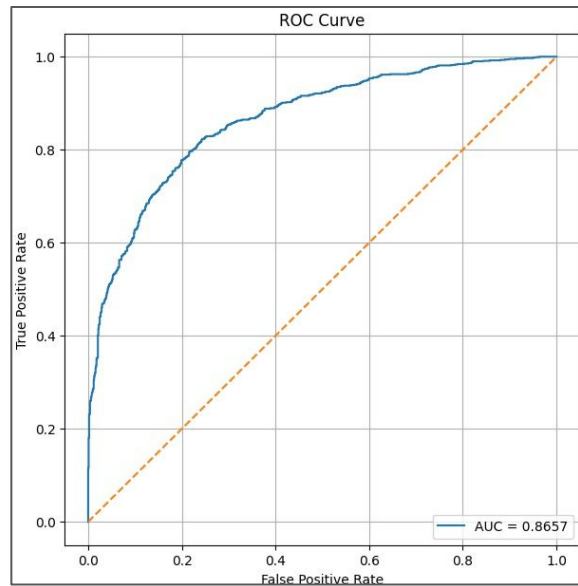
Accuracy Curve

(RGB image - Cat/Dog)



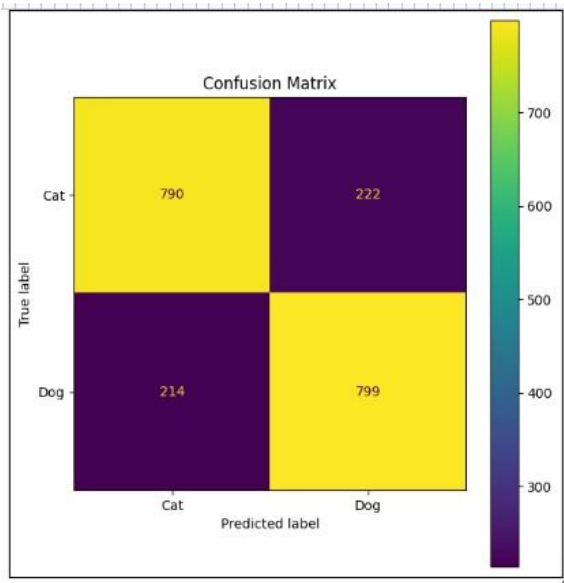
ROC-AUC

(RGB image - Cat/Dog)



Confusion Matrix

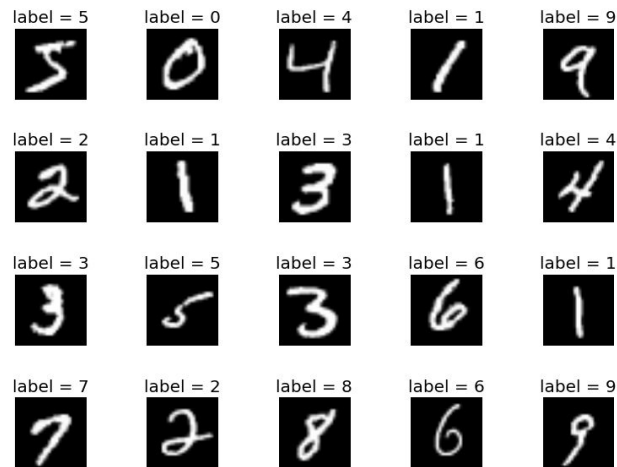
(RGB image - Cat/Dog)



Experiments Performed

(GrayScale Images - MNIST)

In addition to the Cats vs Dogs dataset, the proposed CON-KAN architecture was also experimentally evaluated on the MNIST handwritten digit dataset to observe its behavior on simpler grayscale image classification tasks. The experiments helped analyze the adaptability of the hybrid CNN-KAN architecture across different dataset complexities and demonstrated that KAN-based classification can generalize beyond binary image classification problems.

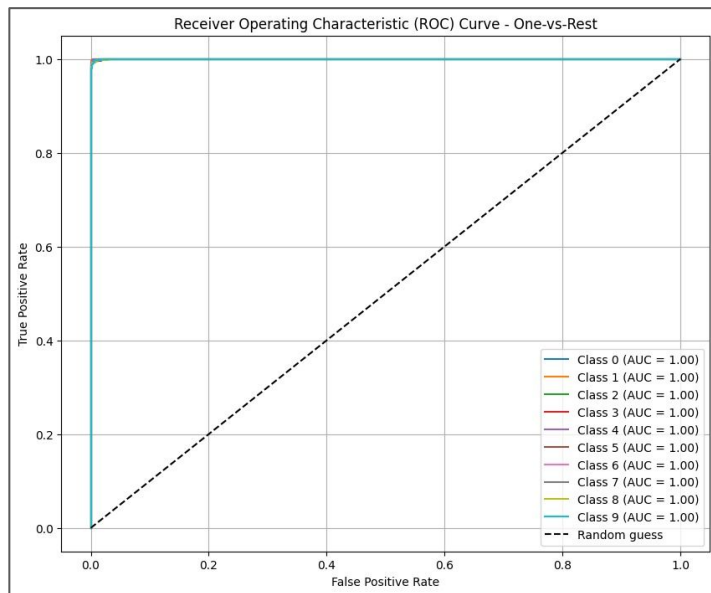


problems:

generalize beyond binary image classification

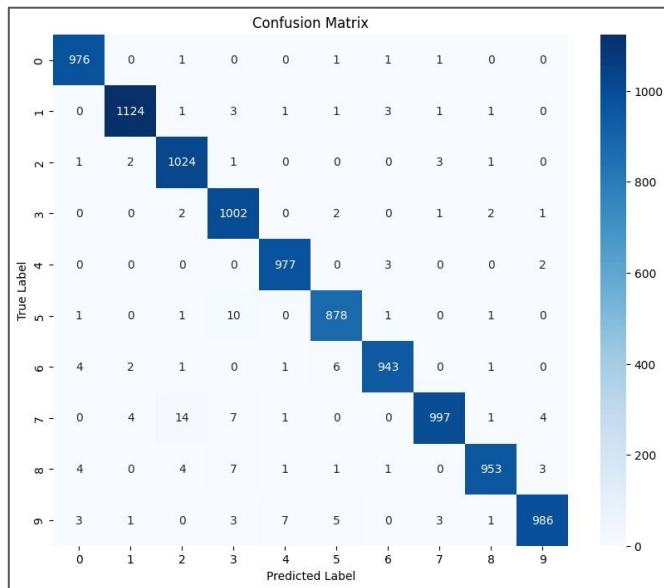
ROC-AUC

(Grayscale image - MNIST)



Confusion Matrix

(Grayscale image - MNIST)



References

- KAN: Kolmogorov–Arnold Networks, Research Paper
 - <https://arxiv.org/pdf/2404.19756>
- Convolutional-KAN
 - <https://share.google/MepTJcKIJ11TvDtch>
- B-Spline On-Line Geometric Modeling Notes, Paper
 - <https://www.cs.unc.edu/~dm/UNC/COMP258/LECTURES/B-spline.pdf>
- B-Spline visualization tool
 - <https://b-spline-site.onrender.com/bspline.html>
- PyTorch
 - <https://pytorch.org/>
- Matplotlib Pyplot
 - https://matplotlib.org/3.5.3/api/as_gen/matplotlib.pyplot.html
- NumPy
 - <https://numpy.org/>
- FastAPI
 - <https://fastapi.tiangolo.com/>
- Pillow
 - <https://pypi.org/project/pillow/>
- Torchvision
 - <https://docs.pytorch.org/vision/stable/index.html>
- Cat Dog Image Dataset
 - <https://www.kaggle.com/datasets/tongpython/cat-and-dog>
- Angular 13
 - <https://v13.angular.io/docs>
- TypeScript
 - <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>
- Font Awesome
 - <http://fontawesome.com/>



Thank you.